

CSC421 Assignment 4

Zili Xie

Mar 25th

Part 1: Deep Convolutional GAN

Implement the Discriminator of DCGAN

1. Padding: number of padding:P, width:W, kernel size:K, stride:S.
In each layer, the widths of input are 32, 16, 8, 4. Also, we know that the relation between width, kernel, padding and stride is: $W_2 = (W_1 - K + 2P)/S + 1$.

layer 1: $16 = (32 - 5 + 2P)/2 + 1$; $P = 1.5$, P round up to be 2.

layer 2: $8 = (16 - 5 + 2P)/2 + 1$; $P = 1.5$, P round up to be 2.

layer 3: $4 = (8 - 5 + 2P)/2 + 1$; $P = 1.5$, P round up to be 2.

layer 4: $1 = (4 - 5 + 2P)/2 + 1$; $P = 0.5$, P round up to be 1.

2. Implementation: completed in cycle_gan.ipynb.

Generator

1. Implementation: completed in cycle_gan.ipynb.

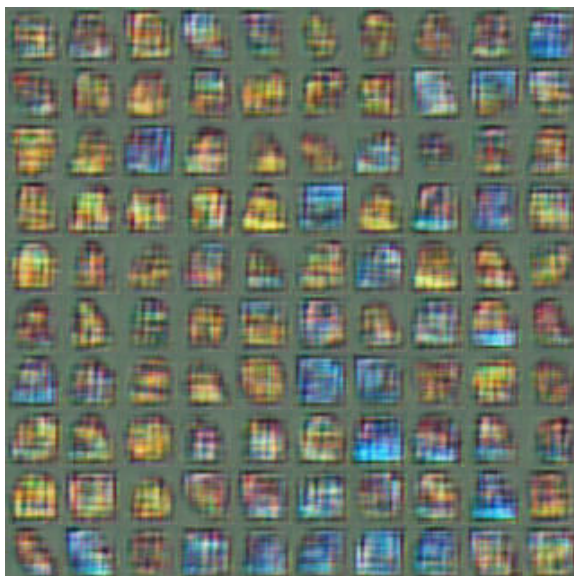
Training Loop

1. Implementation: completed in cycle_gan.ipynb.

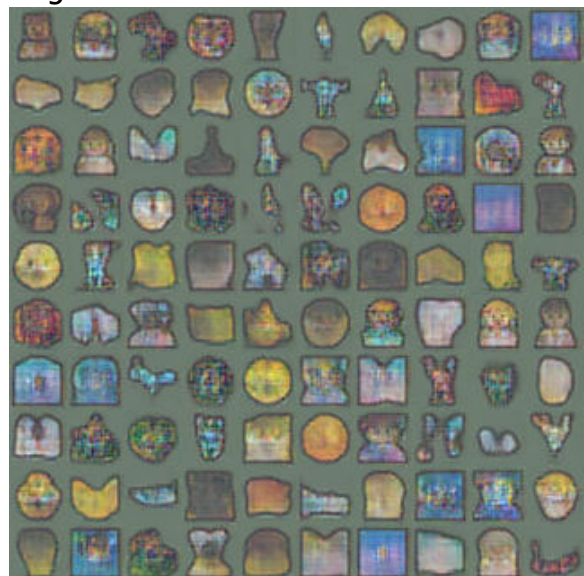
Experiment

1. the image samples after 200 and 5000 iterations are below. The first pair uses emoji from Windows and the second pair uses emoji from Apple.

Windows emoji

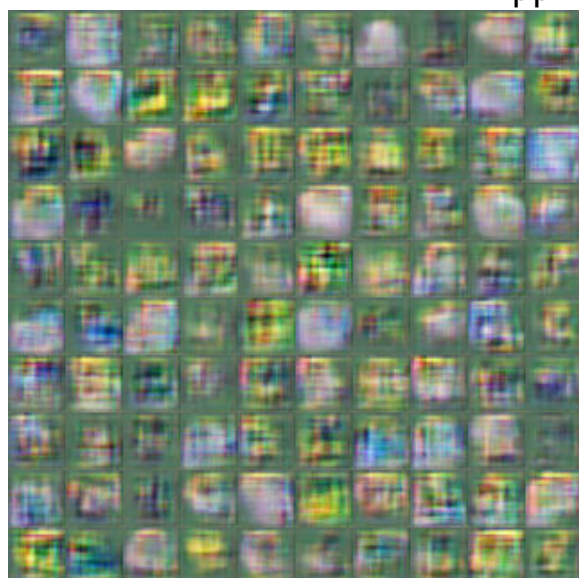


200 iterations

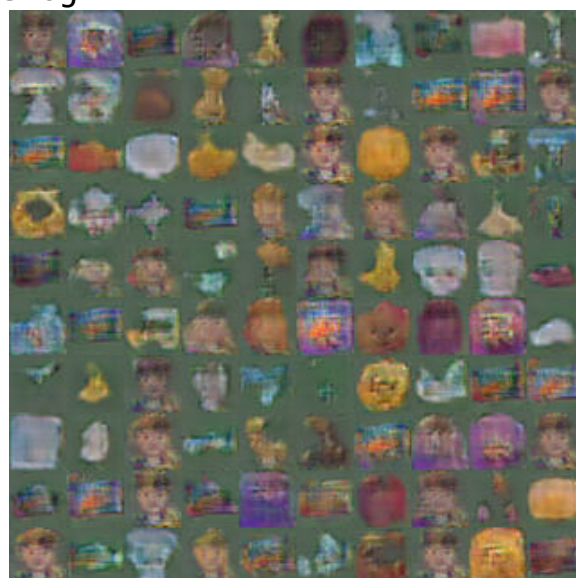


5000 iterations

Apple emoji



200 iterations



5000 iterations

the quality and clarity of the emojis on the right side (5000 iterations) improves a lot comparing to the image generated after 200 iterations. As you can see, the emojis on the left side are more like blurry and mosaicked chips with vertical and horizontal textures. It's impossible to interpret any information about the emoji from the early generated image. But from the 5000 iterations output, we can roughly tell whether an emoji is a human face or national flag or something else.

Windows	200	400	600	800	1000	2000	3000	4000	5000
Emoji1									
Emoji2									
Emoji3									

Apple	200	400	600	800	1000	2000	3000	4000	5000
Emoji1									
Emoji2									

Part 2: CycleGAN

Generator

1. Implementation: completed in `cycle_gan.ipynb`.

CycleGAN Training Loop

1. Implementation: completed in `cycle_gan.ipynb`.

Cycle Consistency

1. Implementation: completed in `cycle_gan.ipynb`.

CycleGAN Experiment

1. the output image from generator that translate Apple emoji to Windows emoji at 200 iterations and 10000 iterations are shown below. The table lists translation details of some emojis at 200th iteration and 10000th iteration respectively.

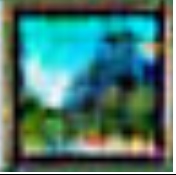


200 iterations



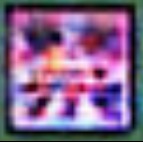
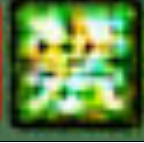
10000 iterations

The table below demonstrate how the generated emojis improve by iterations, as can be seen, the generated emojis from later iteration have higher resolution and sharper edges. The outlines of emoji are certainly more solid and accurate than the generated emoji from iteration 200.

origin	200	5000	10000
			
			
			



2. Training the network with different random seed value will produce emoji with different colorizations. Seed values of 3, 4, 5, 7, 11, 19 are used to train the CycleGAN model, and as it shows below, the color scheme varies when different random seeds are used. All the samples used in this part are extracted from the image output after 10000th iterations to guarantee similar quality. To make it easy to compare, I select three representative emojis: the “number zero”, the “orange” and a Chinese character to demonstrate the difference in color.

Seed	Origin	3	4	5	7	11	19
zero							
orange							
Chinese Character							

Why would this happen? There is no explicit randomness in our CycleGAN model, but the `iter()` function that we use to extract input image actually use random SEED implicitly to select random sample from dataset. As shown in the code block I crop from colab.

```

if iteration % iter_per_epoch == 0:
    iter_X = iter(dataloader_X)
    iter_Y = iter(dataloader_Y)

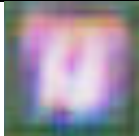
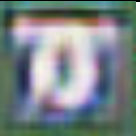
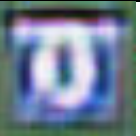
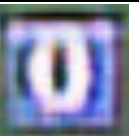
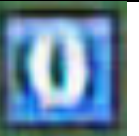
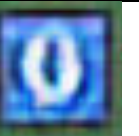
    images_X, labels_X = iter_X.next()
    images_X, labels_X = to_var(images_X), to_var(labels_X).long().squeeze()

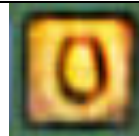
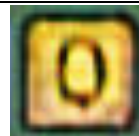
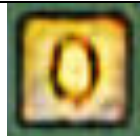
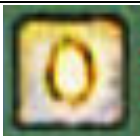
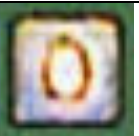
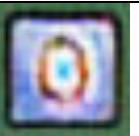
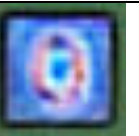
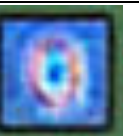
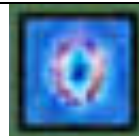
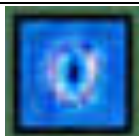
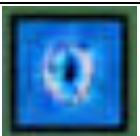
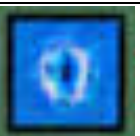
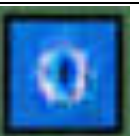
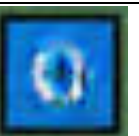
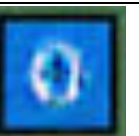
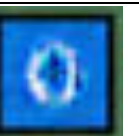
    images_Y, labels_Y = iter_Y.next()
    images_Y, labels_Y = to_var(images_Y), to_var(labels_Y).long().squeeze()

```

So, if we use different random seed values, the emoji images in each batch we used to train our model selected by `iter()` function can be totally different, which may cause our model to be deviated.

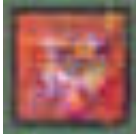
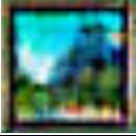
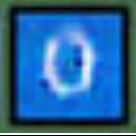
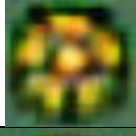
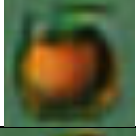
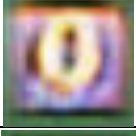
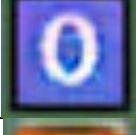
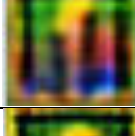
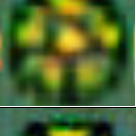
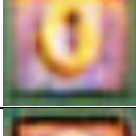
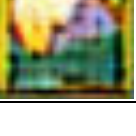
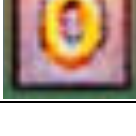
But how could this affect the colorization? My thought is that the colorization of those emojis actually has dependencies with the order of images we input from dataset in early iterations. More specifically, the sample images inputed to train our model can affects “color convergence” time. The model may produce erroneous translation in early stage when only few images are used to train the model. Usually the model will rectify its mistakes in later iterations as the number of images used to train the model increases. But sometimes the model couldn’t correct the mistakes even after 10000 iterations, because it misses the opportunity to “converge”.

Iter	20	40	60	80	100	120	160	200
Seed =4								

Iter	200	400	600	800	1000	1200	1600	2000
Seed =10								
	3000	4000	5000	6000	7000	8000	9000	10000
								

You can see a “color convergence” effect in the two examples above where seed value 4 and 10 are used. I configured the value of “sample_every” to be 20 instead of 200, so that the output can demonstrate this color inversion effect in a better way. Notice that it takes about 200 iterations for model to converge and generate correct colorization if seed value 4 is used. While more than 2000 iterations are needed to product right color if we use random seed value 10 instead of 4. It can even take up to 3000 iterations to totally get rid of incorrect colors around the digit “0”. So obviously, the training image can affect the color converge time, which consequently let the model miss the opportunity to “converge”.

3.
The output images of
all the images below use generation output after 200 iterations
and 10000 iterations with random seed value equals 4 to guarantee
similar quality.

origin								
lambda_cycle = 0.000	200							
	10000							
lambda_cycle = 0.015	200							
lambda_cycle = 0.030	200							
	10000							
lambda_cycle = 0.050	200							
	10000							

If we configure the value of lambda_cycle to be 0, then the generated emojis will lost a lot of details about its inside pattern, which make sense since we don't count the reconstruction loss in g_loss function. Therefore, generator will only receive gradient signals from discriminator, which gives it no enough guidance on how to the feature of the original picture would be. If we increase the lambda_cycle value to 0.030 or 0.050 then the output emojis seems to have colorization problem, but the pattern in each generated emoji is pretty good. That is because we increase the weight of reconstruction loss in G_loss function. So, the model will focus more on reconstructing the pattern.